

SD Standard Subroutines

The 42 catalogued names beginning with `!`, which of them an application may use, and how each is called.

SD ships forty-two catalogued routines whose names begin with `!`. They are in the global catalogue, so any account reaches them without cataloguing anything of its own.

Most exist to support SD itself and are not an interface for applications. They are listed in full because a name in a stack trace, a `map` listing or an error message has to be findable, and because a handful are genuinely useful. Which is which is said plainly below rather than left for you to discover.

These are not a versioned interface. A routine here can change between releases in ways a documented function would not. If you call one, test what it returns rather than assuming its shape holds.

Some are functions and some are subroutines

Get this right first, because calling one the wrong way does not produce a helpful error.

A **subroutine** is reached with `call`, and its results come back in its arguments:

```
call "!ERRTEXT", text, 3035
```

A **function** is declared with `deffun` and returns a value:

```
deffun ftype(path) calling "!FTYPE"  
kind = ftype(@path)
```

`SUBR()` calls a subroutine from an l-type expression in a dictionary.

Nothing here is typed at the TCL prompt. These are called from SD BASIC.

The ones an application may reasonably use

PATHS AND NAMES

<code>!ABSPATH</code>	<code>abspath((dir), rel)</code> — resolve <i>rel</i> against <i>dir</i> and return an absolute path
<code>!PATHTKN</code>	<code>pathtkn((path))</code> — split a path into its components
<code>!FTYPE</code>	<code>ftype(path)</code> — what kind of file is at <i>path</i>
<code>!VALID_OS_NAME</code>	<code>valid_os_name(name)</code> — is <i>name</i> usable as an operating system account name
<code>!VALID_OS_PATH</code>	<code>valid_os_path(path)</code> — is <i>path</i> a well-formed host path
<code>!VALID_SHELL_CMD</code>	<code>valid_shell_cmd(cmd)</code> — is <i>cmd</i> acceptable to hand to a shell

The three validators are the ones SD's own administrative verbs use before handing a value to Windows. If you are building a command line or an account name from input, call them rather than reimplementing the rules.

SESSIONS AND USERS

<code>!USERNAME</code>	<code>username(userno)</code> — the name of the session with that user number
<code>!USERNO</code>	<code>userno((name))</code> — the user number of a named session

ERRORS, FORMATTING AND SORTING

<code>!ERRTEXT</code>	<code>call "!ERRTEXT", expansion, errno</code> — the message text for a status code
<code>!FORMAT</code>	<code>call "!FORMAT", in.rec, out.rec, default.file, indent.step, case.option, errors</code> — the source formatter behind the <code>format</code> verb
<code>!SORT</code>	<code>call "!SORT", in.list, out.list, mode</code> — sort a list
<code>!VOCREC</code>	<code>call "!VOCREC", rec, (id)</code> — build or check a VOC record

`!ERRTEXT` is the useful one. A program that traps a status code and wants to log something a person can read should call it rather than keep its own table of numbers.

SELECTION LISTS ON SCREEN

<code>!PICK</code>	<code>call "!PICK", item, top.line, item.list, title, pick.pos</code> — a scrolling chooser
--------------------	---

<code>!PICKLST</code>	<code>call "!PICK.LIST", value, ...</code> — the list variant
-----------------------	---

Both need a real terminal. Neither does anything useful in a phantom, a scheduled job or an API session.

PRINTING AND PROGRAM INFORMATION

<code>!PCL</code>	<code>pcl(key, arg1 ... arg6)</code> — printer control sequences, variable arguments
-------------------	--

<code>!PROG_INFO</code>	<code>call "!PROG_INFO", obj_dir_path, obj_name, prog_name, prog_obj_sz, prog_comp_time, prog_comp_date, error_msg</code>
-------------------------	---

THE COMMAND PARSER

<code>!PARSER</code>	<code>call "!PARSER", key, type, string, keyword, voc.rec, quote.char</code> — variable arguments
----------------------	---

`!PARSER` is the tokeniser SD's own verbs use to read their arguments, and the keys it takes are defined in `SYSCOM/PARSER.H`. It is the most useful of the internal routines and the one most likely to change.

The internal ones

These exist for SD's own use. They are catalogued because SD's programs are ordinary compiled programs and reach them the same way anything else does.

Calling them from an application is not supported. Several refuse a session that is not elevated or not administrative, and some change the state of the machine.

<code>!ATVAR</code> <code>!SETVAR</code>	read and set the <code>@</code> variables
--	---

<code>!GETPU</code> <code>!SETPU</code>	read and set per-user values
---	------------------------------

<code>!CREATE_USER</code> <code>!DELETE_USER</code> <code>!SET_PASSWD</code>	the Windows account half of <code>create.account</code> , <code>delete.account</code> and <code>modify.password</code>
--	--

<code>!CRED_SET</code> <code>!CRED_VERIFY</code>	write and check a credential in the credential store
--	--

<code>!SD_GET_SALT</code> <code>!SD_KEY_FROM_PW</code>	the key derivation behind that credential
<code>!EUID_SET</code> <code>!EUID_RESTORE</code>	the POSIX effective identity calls
<code>!ELEVATE</code>	starts, uses and stops the elevated helper
<code>!PS_SCRIPT</code> <code>!PS_SCRIPT_OUT</code>	run a PowerShell script through that helper, without and with its output
<code>!IS_USER</code> <code>!IS_GROUP</code> <code>!IS_GRP_MEMBER</code> <code>!IS_SD_USER</code> <code>!OS_GROUP</code>	Windows account and group questions
<code>!PROFILE_DIR</code>	<code>profile_dir(username)</code> — where a Windows profile lives
<code>!SD_ADMIN_TIER</code> <code>!TIER_ALLOWS</code>	the account tier, and whether it permits something
<code>!SDCLIENT</code>	the server side of the client API

What is not here

Documentation for other MultiValue systems describes `!`-prefixed routines SD Core for Windows does not have. Three a reader may come looking for:

<code>!OCONV</code> <code>!ICONV</code>	not catalogued. <code>oconv()</code> and <code>iconv()</code> are ordinary BASIC functions and do the same work
<code>!SCREEN</code>	not catalogued, and there is no equivalent

To settle the question for any name, use `map` in the account. It lists what the catalogue holds. A name that is not there is not available, and calling it fails at run time rather than at compile time.

See also

SD Dictionaries - Conversions and Formatting · SD TCL - Programs and the Catalogue · SD Basic - Program Structure.

